

# Exact Policy-Space Credential Derivation for Failure-Safe Legacy Password Rotation

Yuanyi Zhang

*Independent Researcher*

---

## Abstract

Enterprise networks still contain applications, appliances, and directories whose only modifiable authentication secret is a policy-constrained password. Replacing these systems is preferable, but during migration a manager must generate reconstructible credentials and rotate them through remote interfaces whose success is not always observable. We present Exact Policy-Space Credential Derivation (EPSCD), a construction that compiles a bounded password policy into a finite accepted language, computes its exact cardinality, and builds inverse ranking functions. A context-separated key controls a permutation of the exact rank domain; generation  $g$  is unranked to the credential at that position. The construction specializes established finite-language and rank-then-encipher techniques, giving exact compliance, deterministic reconstruction, exact  $\log_2 |\mathcal{L}_P|$ , and strict non-repetition within one credential lineage. We then couple the sequence to an evidence-bounded rotation protocol. Candidate metadata is durably prepared before submission, and the committed generation advances only when an adapter-specific predicate establishes acceptance of that candidate. Timeouts, lost responses, unsafe verification, and contradictory observations preserve both old and new reconstructability in an unknown-outcome state. In a clean run over the published 270-policy SOUPS corpus, 121 of 121 exactly translated policies compile under a fixed budget. A 100,000-generation run has no policy violation, duplicate, or replay mismatch. 30 fault scenarios across HTTP form and LDAP-style adapter models preserve the two rotation invariants. A bounded TLA+ model explores 1,069 distinct states without invariant violations, while two negative controls are detected. The artifact is a research prototype: its FF1 backend uses bounded cycle walking, its adapter experiments are local semantic models, and it does not make weak legacy policies strong.

*Keywords:* deterministic password generation, password policy, regular language, format-preserving encryption, credential rotation, fault recovery

---

## 1. Introduction

Modern authentication should use phishing-resistant mechanisms when the relying system can be changed. In practice, enterprise networks continue to contain vendor applications, appliances, directories, and internally developed services whose available interface is a password field. Passwords therefore remain a migration and compatibility problem even when an organization has a passwordless target architecture [1]. A management client for these targets must answer two different questions. Which password does generation  $g$  denote under the target policy? When a remote change is attempted, what evidence justifies changing the locally committed generation?

Published deterministic generators demonstrate site-specific derivation without changing the target verifier. PwdHash and Password Multiplier derive a site credential from a master secret and site context [2, 3]; PALPAS separates a high-entropy secret from non-secret per-service state and policy information [4]. Published policy languages and verified generators separately establish that machine-readable constraints and uniform compliant generation are feasible [5, 6]. These are essential precedents, but they do not define a generation-indexed, without-replacement credential sequence coupled to a failure-aware remote password-change journal.

Naively hashing  $(K, \text{service}, g)$  and reducing modulo a legal-space size has two avoidable defects. Reduction can bias the marginal distribution, and independent evaluations can collide at two generations. Constructing required character classes and shuffling with unbiased indices also does not generally make the final distribution uniform over the complete accepted set. The correct abstraction is the finite language accepted by the entire policy. Counting and ranking finite languages, arbitrary-domain ciphers, and rank-then-encipher format-preserving encryption are established results [7, 8, 9, 10]. Our method does not rename these results as new mathematics; it binds them to a credential lineage and generation contract.

Remote rotation presents a separate systems failure. An HTTP success page, a completed TCP exchange, or a submitted LDAP request need not prove that the new credential is the durable remote state. Conversely, a server may commit an update whose response is lost. Advancing local generation

on a transport event can make the manager reconstruct a credential that the target rejects. Rolling back after every timeout can lose a credential that the target accepted. The safe result is sometimes explicitly unknown.

This paper makes exactly two technical contributions:

1. **Exact Policy-Space Credential Sequence.** We specify a bounded policy compiler with exact arbitrary-precision suffix counts, inverse Rank/Unrank functions, an explicit credential lineage, and an audited keyed permutation over  $[0, N_P)$ . The sequence provides exact compliance, deterministic reconstruction, exact effective bits, and strict same-lineage non-repetition. Cross-lineage equality is quantified rather than hidden by a password-history mechanism.
2. **Evidence-Bounded Failure-Safe Legacy Rotation.** We define a durable prepare-submit-verify protocol and an adapter capability contract. Local generation advances only after adapter-defined evidence supports the candidate credential; uncertain outcomes retain both deterministic reconstruction paths and permit later reconciliation.

The contributions are a credential construction and a lifecycle protocol, not a new finite-language algorithm, cryptographic primitive, distributed transaction primitive, or authentication mechanism. EPSCD is intended for legacy compatibility while a target is migrated to modern authentication.

## 2. Related work and novelty boundary

### 2.1. Deterministic and policy-aware credential generation

PwdHash derives site-separated passwords inside a browser extension without server modification [2]. Password Multiplier uses a similar master-secret/site-name model and its evaluation discusses changing the site argument for rotation [3]. PALPAS derives passwords from a high-entropy secret and per-service salts while synchronizing non-secret parameters [4]. These works establish deterministic legacy-verifier compatibility and service separation as prior art.

Horsch et al. define a password-requirements markup language [11]. Gautam et al. design a composition-policy description language and release the 270-policy workbook used in our evaluation [5]. Grilo et al. verify policy compliance and uniform random generation in EasyCrypt and integrate the

generator into Bitwarden [6]. EPSCD neither claims that policy representation nor uniform compliant random generation is new. Its different object is a keyed, deterministic, injective sequence indexed by credential generation.

### *2.2. Finite-language counting and exact-domain permutations*

Uniform generation from formal languages and language ranking have a long literature [12, 13, 14, 15, 10, 7]. We use exact suffix counts and lexicographic unranking, standard dynamic-programming techniques. Dichopile [10] is the published algorithmic comparison in our distribution and cost experiments. The algorithms answer different questions: Dichopile draws independent random words, whereas EPSCD assigns a stable permutation position to each credential generation.

Black and Rogaway study ciphers on arbitrary finite domains [8]. Bellare et al. formalize FPE, analyze cycle walking, and describe rank-then-encipher for complex formats including regular languages [9]. Consequently, the mapping  $\text{Unrank}(\Pi(\text{Rank}(x)))$  is not our invention. EPSCD specializes the approach to credential metadata and rotation. The prototype reuses FF1 and follows the one-million-element minimum-domain direction in the NIST revision draft [16]; it does not propose a cipher.

### *2.3. Key recovery and operational secret handling*

Threshold recovery and multi-factor key derivation are established topics [17, 18, 19]. They are operational dependencies rather than EPSCD contributions. In the intended profile, a complete Root Key is not persistently stored. A managed endpoint retains one Shamir share and removable media retains a second share under password-based authenticated encryption; the password protects the share envelope and does not generate the share. A required operation reconstructs the Root Key transiently, derives the credential key and output, and clears temporary material on a best-effort basis. A third network-resident share is reserved for alternative recovery. This profile stores neither service-password values nor a complete Root Key, but it cannot protect plaintext during a legitimate operation on an already compromised endpoint.

### *2.4. Published-property comparison*

Table 1 reports only properties specified by published work. “Not specified” is not a claim that an implementation cannot be extended.

Table 1: Published work and the two EPSCD research properties.

| Work                                | Deterministic<br>service creden-<br>tial | Exact accepted-<br>space cardinal-<br>ity  | Same-lineage<br>permuted se-<br>quence | Evidence-<br>bounded remote<br>rotation |
|-------------------------------------|------------------------------------------|--------------------------------------------|----------------------------------------|-----------------------------------------|
| PwdHash [2]                         | Yes                                      | Not specified                              | Not specified                          | Not specified                           |
| Password<br>Multiplier [3]          | Yes                                      | Not specified                              | Not specified                          | Not specified                           |
| PALPAS [4]                          | Yes                                      | Not specified                              | Not specified                          | Not specified                           |
| Verified password<br>generation [6] | Random genera-<br>tion                   | Policy semantics<br>verified               | No                                     | No                                      |
| Dichopile [10]                      | No                                       | Exact counting<br>in our reproduc-<br>tion | No                                     | No                                      |
| EPSCD                               | Yes                                      | Yes                                        | Yes                                    | Yes                                     |

### 3. System and threat model

#### 3.1. Entities and persistent state

The client manages a legacy target, a service/account identity, an authenticated policy description, and Root Key material. The target exposes a password-change adapter and possibly a separate verification or version-readback interface. It does not participate in EPSCD and need not store EPSCD metadata.

The baseline persists identifiers, a random credential salt, an explicit lineage identifier, policy identity/version/epoch/hash, root generation, committed credential generation, adapter metadata, and a pending operation journal. It does not persist the current or previous service-password values. During display, submission, and verification, a derived password necessarily exists transiently.

#### 3.2. Adversary and assumptions

The adversary may read public policy and generation metadata; observe known  $(g, pwd_g)$  pairs; steal the metadata database; drop, delay, duplicate, or reorder network traffic; replay a stale local snapshot; and crash the client at a persistence or network boundary. An adapter can be unavailable or return ambiguous observations. We separately discuss adapter compromise.

The authenticated journal is assumed to preserve integrity on the current device. Detecting rollback of the journal itself requires an external or non-rollbackable freshness checkpoint. The selected permutation is assumed to

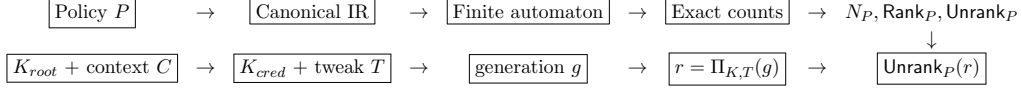


Figure 1: EPSCD data flow. Policy compilation defines one exact finite domain; credential generation is the input to one context-bound permutation, not part of its tweak.

meet its stated PRP security goal on the used domains. Policy and generation metadata are not secret. A fully compromised endpoint during a legitimate derivation can capture the Root Key, credential key, or plaintext password.

### 3.3. Goals and non-goals

The goals are exact policy membership, deterministic reconstruction, same-lineage non-repetition, exact domain measurement, and safe local progress under remote ambiguity. Non-goals include forward secrecy after credential-key compromise, global non-repetition across independent lineages, availability against a malicious operator, complete constant-time behavior, and making a small accepted language resistant to exhaustive search.

## 4. Exact Policy-Space Credential Sequence

### 4.1. Bounded policy language

Let  $P$  be a canonical bounded policy over alphabet  $\Sigma$ , with lengths in  $[L_{\min}, L_{\max}]$ . The implemented fragment supports allowed and forbidden characters; minimum and maximum class counts; fixed positions, prefix, and suffix; forbidden first/last sets; a per-character maximum; identical and sequential run limits; and finite forbidden substrings. Their product state is a finite deterministic transition system

$$A_P = (Q, \Sigma, \delta, q_0, F).$$

The exact accepted language is

$$\mathcal{L}_P = \{w \in \Sigma^* : L_{\min} \leq |w| \leq L_{\max}, \delta^*(q_0, w) \in F\}.$$

Unsupported source semantics are rejected, not approximated. The compiler also enforces a state budget because the product of class counters, substring state, run state, and position constraints can be exponential in the policy description.

#### 4.2. Exact suffix counts

For position  $i$ , target length  $\ell$ , and automaton state  $q$ , define

$$C_\ell(i, q) = \begin{cases} 1, & i = \ell \wedge q \in F, \\ 0, & i = \ell \wedge q \notin F, \\ \sum_{c \in \Sigma: \delta(q, c) \downarrow} C_\ell(i+1, \delta(q, c)), & i < \ell. \end{cases}$$

All counts use arbitrary-precision integers. The exact domain size is

$$N_P = |\mathcal{L}_P| = \sum_{\ell=L_{\min}}^{L_{\max}} C_\ell(0, q_0), \quad H_{\text{eff}} = \log_2 N_P.$$

For  $Q = |Q|$ , maximum length  $L$ , and alphabet size  $s$ , a dense suffix table uses  $O(LQ)$  big integers and  $O(LQs)$  transition/count operations after the automaton is constructed. Big-integer bit complexity depends on  $\log N_P$ . The prototype reports state count, count payload, process RSS, compile time, and online rank/unrank latency rather than hiding these costs in one average.

#### 4.3. Ranking and unranking

Fix a canonical order on lengths and on  $\Sigma$ . Unranking first selects the length interval containing rank  $r$ . At each position, it considers outgoing characters in canonical order; the completion count following each edge is an interval width. It selects the interval containing  $r$  and subtracts preceding widths. Ranking applies the inverse accumulation to an accepted word.

**Theorem 1** (Bijection). *For a successfully compiled non-empty policy,  $\text{Rank}_P : \mathcal{L}_P \rightarrow [0, N_P)$  and  $\text{Unrank}_P : [0, N_P) \rightarrow \mathcal{L}_P$  are mutual inverses.*

*Proof.* At each prefix, outgoing edge intervals are disjoint, adjacent, and their widths sum to the suffix count of the current state by the recurrence. Therefore every rank selects exactly one edge and every accepted prefix contributes exactly the sum of intervals preceding its edge. Induction over remaining positions gives  $\text{Rank}_P(\text{Unrank}_P(r)) = r$ . Applying the same argument to the unique path of an accepted word gives  $\text{Unrank}_P(\text{Rank}_P(w)) = w$ .  $\square$

The decoder performs no random candidate rejection. This statement does not apply to the concrete permutation layer.

#### 4.4. Context-separated exact-domain sequence

Scheme v2 persists a public random *lineageID*. A credential key is derived using canonical length-delimited metadata:

$$K_{\text{cred}} = \text{HKDF}(K_{\text{root}}, \text{EPSCD/credential-key/scheme-v2} \parallel \\ \text{vaultID} \parallel \text{serviceID} \parallel \text{accountID} \parallel \text{lineageID} \parallel \text{credentialSalt} \parallel \\ \text{rootGeneration} \parallel \text{policyVersion} \parallel \text{policyEpoch}).$$

The tweak additionally binds the policy identifier and canonical policy hash. Generation is deliberately the permutation input, not part of the tweak:

$$r_g = \Pi_{K_{\text{cred}}, T}(g), \quad \text{pwd}_g = \text{Unrank}_P(r_g), \quad 0 \leq g < N_P.$$

The concrete backend applies AES-256 FF1 to the smallest binary domain  $[0, M)$  containing  $[0, N_P)$  and cycle-walks until the result is below  $N_P$ . It fails closed after 1,024 walks, supports domains of at least one million elements and at most 512 bits, and never falls back to modulo reduction. Since  $M < 2N_P$ , the accepted density is greater than one half for this selected superset. The implementation is not claimed to be constant time.

**Theorem 2** (Exact compliance and deterministic reconstruction). *Every successful derivation returns an element of  $\mathcal{L}_P$ . Identical root, canonical context, policy, generation, and backend produce the same password.*

*Proof.* The backend returns  $r_g \in [0, N_P)$  and Theorem 1 establishes that  $\text{Unrank}_P(r_g) \in \mathcal{L}_P$ . All transformations are deterministic for fixed inputs.  $\square$

**Theorem 3** (Same-lineage non-repetition). *For fixed key, tweak, and policy, successful derivations at  $g_1 \neq g_2$  return different passwords until the domain is exhausted.*

*Proof.*  $\Pi_{K, T}$  is injective and  $\text{Unrank}_P$  is injective. Their composition is injective.  $\square$

**Theorem 4** (Ideal-permutation marginal). *For a fixed generation and a uniformly selected ideal random permutation key, each  $w \in \mathcal{L}_P$  occurs with probability  $1/N_P$ .*

*Proof.* The image of a fixed input under an ideal random permutation is uniform on  $[0, N_P)$ . Bijection under  $\text{Unrank}_P$  preserves the mass of every element.  $\square$



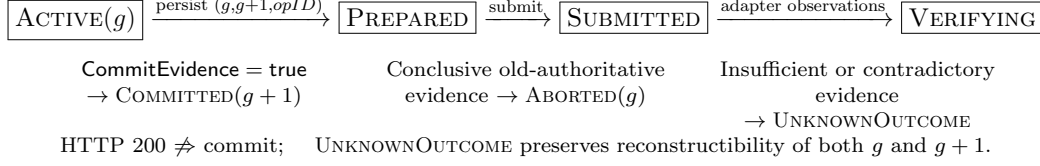


Figure 2: Evidence-bounded remote rotation. Persisting a candidate and sending an update never commits local metadata. The adapter contract determines whether new acceptance, new-only acceptance, atomic evidence, or authoritative version readback is sufficient.

For the concrete backend, Theorem 4 is replaced by the applicable computational PRP claim. Known  $(g, \text{pwd}_g)$  pairs are known input/output pairs after ranking and do not provide forward secrecy.

#### 4.5. Lineage change and collision bound

Rekeying, root replacement, or a semantically changed policy starts a new lineage. Strict non-repetition is intentionally scoped to one lineage. Let  $H$  be historical passwords,  $\mathcal{L}_{\text{new}}$  the new language,  $h = |H \cap \mathcal{L}_{\text{new}}|$ , and  $N = |\mathcal{L}_{\text{new}}|$ . Under an independent ideal permutation, the next output collides with this relevant history with probability  $h/N$ . For  $m$  future outputs, a simple union bound is

$$\Pr[\text{collision}] \leq \min\left(1, \frac{hm}{N}\right).$$

The prototype returns this bound as an exact rational. It stores no password history in the baseline. A deployment can configure a minimum  $H_{\text{eff}}$  and reject or warn on a weak legacy policy.

## 5. Evidence-Bounded Failure-Safe Rotation

### 5.1. Why remote success is an evidence question

A remote password-change operation crosses independent persistence and network boundaries. The request may never arrive; the server may commit and lose its response; a proxy may return a success page before durable update; replicated authentication nodes may temporarily disagree; or verification may be unsafe because failed attempts consume a lockout budget. Therefore no universal mapping from transport status to committed credential exists.

### 5.2. Adapter capability and evidence contract

Each adapter declares whether it can verify the new credential, safely verify the old credential, obtain atomic success evidence, read an authoritative remote version, and attach an idempotency key. Its evidence requirement is one of:

- **NEWACCEPTANCE**: a conclusive candidate probe is sufficient;
- **NEWONLY**: the candidate succeeds and a safe old probe conclusively fails;
- **AUTHORITATIVEVERSION**: an expected remote version/readback is observed;
- **UNKNOWNONLY**: automatic commit evidence cannot be established.

An adapter-specific predicate **CommitEvidence**( $A, O$ ) evaluates declared capabilities  $A$  and persisted observations  $O$ . A generic update success flag is recorded but ignored by the predicate. The old probe is omitted when unsafe; this can narrow the conclusion to new acceptance or force an unknown outcome rather than spending the lockout budget.

LDAP provides a standardized password-modify operation [20], but an adapter must still define what a successful operation and subsequent readback mean in its deployment. The protocol does not treat the existence of a standard operation as universal evidence of replicated authentication state.

### 5.3. Durable transition system

Let  $g$  be the committed generation. Before any possible network send, the client persists

$$\text{PREPARED}(g, g + 1, \text{opID}, \text{lineage}, \text{policy}, A).$$

The record contains metadata sufficient to reconstruct both candidates, not their password values. Submission enters **SUBMITTED**; observations enter **VERIFYING**. The only terminal decisions are:

|                              |                                                 |
|------------------------------|-------------------------------------------------|
| <b>COMMITTED</b> ( $g + 1$ ) | if <b>CommitEvidence</b> ( $A, O$ ),            |
| <b>ABORTED</b> ( $g$ )       | if evidence conclusively retains the old state, |
| <b>UNKNOWNOUTCOME</b>        | otherwise.                                      |

UNKNOWNOUTCOME neither increments nor automatically rolls back. It retains the base generation, candidate generation, operation identity, and context needed to reconstruct both. Later reconciliation adds observations to the journal. A stale operation identifier cannot commit a different candidate.

**Theorem 5** (Commit safety). *Assuming authenticated journal writes and correct adapter classification, the committed generation cannot advance from  $g$  to  $g + 1$  without persisted evidence that satisfies the declared adapter predicate for the corresponding candidate.*

*Proof.* The prepare, submit, verify, crash, unknown, and abort transitions do not modify the committed generation. The only transition that does is commit. Its guard requires the persisted candidate identity and a true value of the adapter evidence predicate. Induction over a transition trace establishes the invariant.  $\square$

**Theorem 6** (Uncertainty preservation). *Every reachable UNKNOWNOUTCOME record retains sufficient authenticated metadata to reconstruct both  $pwd_g$  and  $pwd_{g+1}$ .*

*Proof.* Preparation persists both generation contexts before submission. Every transition into UNKNOWNOUTCOME preserves that record; deletion of either path is permitted only by a terminal commit or abort transition.  $\square$

These are safety results under the stated model, not claims of liveness or a new distributed transaction primitive.

## 6. Implementation

The Rust prototype uses a canonical JSON policy IR, a reachable-state compiler, and a dense arbitrary-precision suffix-count table. The implementation caps reachable states at 250,000 and policy-domain length at the documented bound. Rank and unrank scan canonical outgoing characters; policy acceptance reuses the compiled transitions. HKDF-SHA-256 derives scheme-v2 credential keys. The permutation dependency implements FF1 with AES-256; EPSCD supplies range, domain, and cycle-walk guards.

Rotation metadata is held in authenticated credential-description records and a SQLite store. The adapter contract is a typed structure and the evidence classifier returns sufficient-new-acceptance, sufficient-new-only, sufficient-atomic, sufficient-version, insufficient, or contradictory. The local semantic

adapters reopen an on-disk SQLite journal during crash cases and contain no password column.

The artifact command runs Rust unit/property tests, the public-corpus compiler, controlled-density and sequence experiments, rotation faults, and TLA+, then generates JSON, CSV, and LaTeX tables. Paper numbers in this section are inputs from those generated tables.

## 7. Evaluation

We ask six questions. RQ1 measures realistic policy coverage and compiler cost; RQ2 checks exactness; RQ3 studies the cost of whole-candidate rejection as acceptance density falls; RQ4 exercises a long deterministic sequence; RQ5 tests failure safety; and RQ6 checks whether different adapter capabilities fit one lifecycle.

### 7.1. Setup and measurement boundaries

The policy corpus is the public historical workbook published with Gautam et al. [5]. It is a reproducibility corpus, not a claim about current enterprise prevalence. Translation rejects unsupported semantics instead of weakening them. Each translated policy receives fixed bounds of 250,000 states, 4 GiB RSS, and 60 seconds. Timing values are single-host prototype measurements. The published Dichopile algorithm is reproduced with exact BigUint arithmetic; we do not claim performance against the authors' optimized experimental configuration.

### 7.2. RQ1: policy coverage and exact-space cost

Table 2 shows all 270 source records. Of these, 121 can be translated exactly into the implemented bounded IR and 149 are rejected for unsupported source semantics. Of the attempted exact translations, 121 compile, with 0 reaching the fixed time limit. Among successful policies, median/P95 reachable states are 65/508 and maximum is 49401. Median/P95 count payload is 0.48/18.75 MiB; maximum is 439.40 MiB. Sampled process RSS remains in the machine-readable record, with the stated warning that short-lived peaks can be missed. These tails justify reporting resource limits rather than only a success percentage.

Median policy-level rank and unrank latencies are 23.57 and 33.19  $\mu$ s. On the separate 18-character performance policy, cached full derivation is 128.90  $\mu$ s median and 242.54  $\mu$ s P95; cold compile-and-derive is 5.53 ms

Table 2: Public policy corpus clean run.

| Metric                   | Result |
|--------------------------|--------|
| Source policies          | 270    |
| Exact translations       | 121    |
| Compiled                 | 121    |
| Time limit               | 0      |
| Median states            | 65     |
| P95 states               | 508    |
| Median compile (ms)      | 42.73  |
| P95 compile (ms)         | 694.99 |
| Median rank ( $\mu$ s)   | 23.57  |
| Median unrank ( $\mu$ s) | 33.19  |

median. Compiler cost is therefore paid at policy enrollment or change, not on each routine credential reconstruction.

### 7.3. RQ2: exactness and distribution checks

Small-language tests enumerate every rank and verify acceptance. They check

$$\text{Rank}(\text{Unrank}(r)) = r \quad \text{and} \quad \text{Unrank}(\text{Rank}(w)) = w.$$

Property tests vary length and class requirements. A count exceeding  $2^{128}$  verifies that cardinality is not truncated to a machine integer. Permutation tests check inversion and injection on sampled inputs.

For the enumerable language  $\{a, b0, \dots, b9\}$ , 100,000 independent samples from the exact Dichopile reproduction have empirical total-variation distance 0.00399 from uniform; the test-only ideal-permutation EPSCD composition has distance 0.00234. These are finite-sample sanity checks, not proofs and not a concrete FF1 security experiment.

### 7.4. RQ3: controlled rejection density

For independent whole-candidate rejection with acceptance density  $\alpha$ , the retry count  $R$  is geometric:

$$\mathbb{E}[R] = 1/\alpha, \quad \Pr[R > t] = (1 - \alpha)^t.$$

Table 3 demonstrates the increasingly long online tail. At  $\alpha = 10^{-4}$ , the observed median is 6,696 attempts and P99.9 is 75,963. EPSCD’s language

Table 3: Whole-candidate rejection retries, 2,000 accepted outputs per density.

| $\alpha$ | $1/\alpha$ | Median | P95   | P99   | P99.9 |
|----------|------------|--------|-------|-------|-------|
| 1e-01    | 10         | 7      | 28    | 40    | 60    |
| 1e-02    | 100        | 69     | 291   | 449   | 933   |
| 1e-03    | 1000       | 661    | 3012  | 4713  | 7427  |
| 1e-04    | 10000      | 6696   | 31022 | 45236 | 75963 |

Table 4: Scheme-v2 long-sequence implementation check.

| Metric              | Result  |
|---------------------|---------|
| Generations         | 100,000 |
| Policy violations   | 0       |
| Duplicate passwords | 0       |
| Replay mismatches   | 0       |
| Effective bits      | 70.99   |
| Derivations/s       | 8493    |

decoder follows one count-selected path and therefore has no candidate-retry tail. This comparison isolates the mechanism; the selected FF1 backend has its own bounded cycle walks and must not be described as wholly rejection-free.

Across the 121 compiled corpus policies, 97 satisfy the FF1 backend’s domain bounds and 24 are rejected by that backend while remaining compiler successes. For 3,104 measured generations, cycle walks have median 1, P95 3, maximum 9, and 0 cap failures. Because the smallest binary superset has density above one half, these observations are not the same experiment as the synthetic  $10^{-4}$  whole-candidate density.

#### 7.5. RQ4: sequence behavior

The run derives and replays 100,000 consecutive generations in a domain of 2,351,927,613,440,572,602,240 elements (70.99 effective bits). Table 4 reports no policy violation, duplicate, or replay mismatch. The run takes 23.55 seconds for 200,000 derivations on the test host. This validates the implementation path at scale; non-repetition itself follows from Theorem 3 rather than empirical collision absence.

Table 5: Fault matrix across two adapter semantics.

| Metric             | Result |
|--------------------|--------|
| Adapters           | 2      |
| Injected scenarios | 30     |
| Committed          | 15     |
| Unknown outcome    | 11     |
| Aborted            | 2      |
| All safety checks  | pass   |

#### 7.6. *RQ5–RQ6: faults and adapter diversity*

The matrix covers request drop, remote success plus response loss, time-out, duplicate submission, crashes before and after submission, crash after remote commit, old/new/both/neither acceptance, policy change, lockout risk, unavailable old verification, and absent authoritative readback. The HTTP form adapter uses new-acceptance evidence and never probes the old password. The LDAP-style directory adapter uses expected-version evidence and idempotent operation identity. They share one lifecycle but reach different safe terminal states.

All 30 scenarios satisfy the commit-evidence and uncertainty-preservation checks. 15 reach committed because the configured adapter predicate is satisfied; 11 remain unknown, 2 remain prepared after a pre-submit crash, and 2 abort after the directory model conclusively observes the base version. The experiment uses high-fidelity local semantic models and real durable SQLite reopen, but not a production HTTP application or OpenLDAP deployment.

#### 7.7. *Bounded model checking*

The dedicated model represents four adapter evidence requirements, four remote credential states, persistent preparation, crashes, unknown outcomes, and sequential generations.

TLC exhaustively explores the configured finite state graph. Invariants cover commit evidence, remote candidate acceptance, uncertainty reconstruction, prepare-before-submit, no advance in unknown state, and sequential generation uniqueness. The two negative controls enable a transition that commits on a generic successful response and a transition that drops the candidate on unknown outcome; the expected invariants are violated. The result

Table 6: TLC results for the EPSCD rotation abstraction.

| Metric                     | Result |
|----------------------------|--------|
| Generated states           | 2,877  |
| Distinct states            | 1,069  |
| Search depth               | 16     |
| Main invariant violations  | 0      |
| Negative controls detected | 2/2    |

validates the abstract transition relation within its bounds and assumptions, not the complete Rust implementation or an unbounded network.

## 8. Security analysis

*Known credential pairs..* Generation is public. Given  $(g_i, pwd_{g_i})$ , an observer can compute  $(g_i, \text{Rank}_P(pwd_{g_i}))$ , a known input/output pair for the permutation. Security of unseen generations is the selected PRP’s computational claim, not information-theoretic secrecy or forward secrecy.

*Credential-key and Root-Key compromise..* Compromise of  $K_{\text{cred}}$  exposes its complete lineage. Replacing only the policy epoch does not recover from that exposure because the epoch is public context; a new credential salt and lineage identifier are required. Root-Key compromise affects every derived credential context. Shamir resharing of the same root changes share material but does not recover from root compromise.

*Metadata theft..* Policy, salt, generation, lineage, and adapter metadata are non-secret. Database theft reveals account relationships and enables rollback attempts, but does not alone yield the credential key under the root-secret assumption. Metadata integrity and freshness remain necessary because changing generation or policy context changes the reconstructed password.

*Low-entropy accepted language..* Uniformity cannot repair a small verifier domain. EPSCD reports exact effective bits and can enforce a deployment threshold. The one-million-element FF1 floor is an implementation/backend limit, not a sufficient password-strength rule.



*Endpoint and adapter compromise..* An endpoint compromised during legitimate reconstruction can observe plaintext and key material. A compromised adapter can forge observations within its trust boundary and defeat commit classification. Deployment must authenticate adapter code and evidence sources; the state machine prevents accidental inference, not malicious evidence by a trusted component.

*Rollback and stale state..* The journal prevents ordinary crash loss when the latest authenticated record is available. A complete rollback of both local state and its freshness anchor is outside that guarantee. An independent checkpoint can detect stale generation and pending-operation metadata, but such a service is not required for the two core constructions and is not evaluated as a contribution here.

*Remote ambiguity and availability..* Unknown outcome is a deliberate safety state. It may require operator action or a later authoritative observation, and a target that exposes no safe evidence may remain unresolved indefinitely. This sacrifices automatic liveness rather than guessing a credential state.

## 9. Limitations and deployment guidance

The policy compiler supports a substantial bounded fragment, not arbitrary natural-language policies or breach-list membership. Product-specific semantic translation remains a trusted step. State construction can grow exponentially, so fixed state, memory, and time budgets remain necessary even when all exact translations complete within the final evaluation budget.

The FF1 dependency and cycle-walking wrapper are a prototype backend. The 1,024-walk cap creates explicit failure for rare inputs rather than total derivation for every possible key/tweak/input. Timing varies with walk count and the complete implementation is not constant time. A production deployment would require independent cryptographic review and side-channel analysis.

The adapter evaluation does not include a real enterprise application, OpenLDAP cluster, replication lag, CAPTCHA/MFA workflow, or measured human reconciliation. The capability model is designed to express these differences, but production evidence predicates must be validated per target. The formal model is bounded and checks safety, not liveness.

Frequent forced password rotation is not itself recommended. EPSCD supports necessary compromise response, inherited policy, and migration

workflows; it should not be used to justify needless rotation or to delay replacement of password-only authentication.

## 10. Conclusion

EPSCD treats the complete bounded policy as the credential domain and treats remote change success as an evidence decision. Exact counting and inverse ranking expose the true domain; a context-separated exact-domain permutation turns generation into a deterministic, same-lineage non-repeating sequence. Durable preparation and adapter-specific evidence prevent local generation from advancing on ambiguous transport outcomes while retaining both recovery paths. The implementation, public-corpus run, fault matrix, and bounded model checking support these scoped claims. The method composes established mathematical and cryptographic tools into a legacy credential lifecycle; it does not claim new primitives or remove the need to migrate legacy targets.

## Artifact availability

The artifact contains source, tests, the public-corpus translation provenance, machine-readable results, generated tables, and a one-command reproduction script. Identifying repository information can be replaced with an anonymous artifact URL during double-blind review.

## References

- [1] C. Herley, P. C. van Oorschot, A research agenda acknowledging the persistence of passwords, *IEEE Security & Privacy* 10 (1) (2012) 28–36. doi:10.1109/MSP.2011.150.
- [2] B. Ross, C. Jackson, N. Miyake, D. Boneh, J. C. Mitchell, Stronger password authentication using browser extensions, in: 14th USENIX Security Symposium, USENIX Association, 2005, pp. 17–32. URL [https://www.usenix.org/legacy/events/sec05/tech/full\\_papers/ross/ross.pdf](https://www.usenix.org/legacy/events/sec05/tech/full_papers/ross/ross.pdf)
- [3] J. A. Halderman, B. Waters, E. W. Felten, A convenient method for securely managing passwords, in: *Proceedings of the 14th International Conference on World Wide Web*, ACM, 2005, pp. 471–479. doi:10.1145/1060745.1060815.

- [4] M. Horsch, A. H<sup>o</sup>ulsing, J. Buchmann, PALPAS—passwordless password synchronization, in: 10th International Conference on Availability, Reliability and Security, IEEE, 2015, pp. 30–39. doi:10.1109/ARES.2015.23.
- [5] A. Gautam, S. Lalani, S. Ruoti, Improving password generation through the design of a password composition policy description language, in: Eighteenth Symposium on Usable Privacy and Security, USENIX Association, 2022, pp. 541–560.  
URL <https://www.usenix.org/conference/soups2022/presentation/gautam>
- [6] M. Grilo, J. Campos, J. F. Ferreira, J. B. Almeida, A. Mendes, Verified password generation from password composition policies, in: Integrated Formal Methods, Lecture Notes in Computer Science, Springer, 2022, pp. 271–288. doi:10.1007/978-3-031-07727-2\_15.
- [7] A. V. Goldberg, M. Sipser, Compression and ranking, in: 17th Annual ACM Symposium on Theory of Computing, ACM, 1985, pp. 440–448. doi:10.1145/22145.22193.
- [8] J. Black, P. Rogaway, Ciphers with arbitrary finite domains, in: Topics in Cryptology—CT-RSA 2002, Vol. 2271 of Lecture Notes in Computer Science, Springer, 2002, pp. 114–130. doi:10.1007/3-540-45760-7\_9.
- [9] M. Bellare, T. Ristenpart, P. Rogaway, T. Stegers, Format-preserving encryption, in: Selected Areas in Cryptography, Vol. 5867 of Lecture Notes in Computer Science, Springer, 2009, pp. 295–312. doi:10.1007/978-3-642-05445-7\_19.
- [10] J. Oudinet, A. Denise, M.-C. Gaudel, A new dichotomic algorithm for the uniform random generation of words in regular languages, Theoretical Computer Science 502 (2013) 165–176. doi:10.1016/j.tcs.2012.07.025.
- [11] M. Horsch, M. Schlipf, J. Braun, J. Buchmann, Password requirements markup language, in: Information Security and Privacy, Lecture Notes in Computer Science, Springer, 2016, pp. 426–439. doi:10.1007/978-3-319-40253-6\_26.

- [12] T. Hickey, J. Cohen, Uniform random generation of strings in a context-free language, *SIAM Journal on Computing* 12 (4) (1983) 645–655. doi:10.1137/0212044.
- [13] M. Goldwurm, Random generation of words in an algebraic language in linear binary space, *Information Processing Letters* 54 (4) (1995) 229–233. doi:10.1016/0020-0190(95)00025-8.
- [14] A. Denise, P. Zimmermann, Uniform random generation of decomposable structures using floating-point arithmetic, *Theoretical Computer Science* 218 (2) (1999) 233–248. doi:10.1016/S0304-3975(98)00323-5.
- [15] O. Bernardi, O. Giménez, A linear algorithm for the random sampling from regular languages, *Algorithmica* 62 (1–2) (2012) 130–145. doi:10.1007/s00453-010-9446-5.
- [16] National Institute of Standards and Technology, Methods for format-preserving encryption: SP 800-38G revision 1, second public draft, Tech. rep., NIST (2025).  
URL <https://csrc.nist.gov/pubs/sp/800/38/g/r1/2pd>
- [17] A. Shamir, How to share a secret, *Communications of the ACM* 22 (11) (1979) 612–613. doi:10.1145/359168.359176.
- [18] V. Nair, D. Song, Multi-factor key derivation function (MFKDF) for fast, flexible, secure, & practical key management, in: 32nd USENIX Security Symposium, USENIX Association, 2023, pp. 2097–2114.  
URL <https://www.usenix.org/conference/usenixsecurity23/presentation/nair-mfkdf>
- [19] M. Scarlata, M. Backendal, M. Haller, MFKDF: Multiple factors knocked down flat, in: 33rd USENIX Security Symposium, USENIX Association, 2024, pp. 4301–4318.  
URL <https://www.usenix.org/conference/usenixsecurity24/presentation/scarlata>
- [20] K. Zeilenga, LDAP password modify extended operation, RFC 3062 (2001). doi:10.17487/RFC3062.  
URL <https://www.rfc-editor.org/rfc/rfc3062.html>